

## **4. Test Management**

- 4.1. Test Organization- Roles & Skills of Tester, Test Lead, Test Manager
- 4.2. Test Planning- Test Plan as per IEEE 829 STANDARD TEST PLAN TEMPLATE
- 4.3. Test Process Monitoring & Control
  - 4.3.1. Test Monitoring through -Test Log (IEEE 829: TEST LOG TEMPLATE) and Defect Density
  - 4.3.2. Reporting Test Status (IEEE 829: TEST SUMMARY REPORT TEMPLATE)
  - 4.3.3 Test Control
- 4.4. Test Scenario, Test Suite, Test Cases (both Positive & Negative Test Cases, as per IEEE 829: TEST CASE SPECIFICATION TEMPLATE)
- 4.5. Configuration Management- Configuration Management support for Testing
- 4.6. Risk and Testing- Project Risk & Product Risk
- 4.7. Incident/ Defect Management
  - 4.7.1. Defect Life Cycle
  - 4.7.2. Defect/ Incident Report (IEEE 829: TEST INCIDENT REPORT TEMPLATE )

Case Study on Test Plan for applications and Case study on Test Cases for different features within applications

## 4.1. Test Organization- Roles & Skills of Tester, Test Lead, Test Manager

### 1. Tester (Test Engineer):

- **Roles:**
  - Understand the project requirements and what needs to be tested.
  - Create test cases — detailed steps to check if a feature works as expected.
  - Execute tests manually or through automation tools.
  - Identify bugs or defects and report them clearly to developers.
  - Re-test fixed bugs to ensure the issues are resolved.
  - Perform different types of testing like functional, regression, and performance testing.
- **Skills:**
  - Basic understanding of software development and testing concepts.
  - Attention to detail to catch even small bugs.
  - Problem-solving skills to figure out why something isn't working.
  - Familiarity with tools like JIRA, Selenium, or TestRail.
  - Basic programming knowledge can be helpful for automation testing.

### 2. Test Lead:

- **Roles:**
  - Plan and organize the testing activities for the project.
  - Assign tasks to testers and ensure they understand their work.
  - Prepare test plans and strategies, outlining what needs to be tested and how.
  - Monitor the progress of testing and address any roadblocks.
  - Review test cases and bug reports to ensure they are thorough and clear.
  - Communicate with developers and project managers about testing status.
- **Skills:**
  - Strong leadership and communication skills to guide the team.
  - Good understanding of testing processes and methodologies.

- Ability to manage time and prioritize tasks.
- Knowledge of test management tools and reporting.
- Risk assessment skills to identify potential issues early.

### 3. Test Manager:

- **Roles:**

- Define the overall testing objectives and scope for the project.
- Manage the testing team's resources, timelines, and budgets.
- Coordinate with stakeholders, developers, and clients to align testing efforts with business goals.
- Ensure the testing process follows company and industry standards.
- Analyze test results and prepare reports for senior management.
- Identify risks in the project and create plans to mitigate them.

- **Skills:**

- Strong planning and organizational skills.
- Excellent communication and decision-making abilities.
- Budget management and resource allocation knowledge.
- Understanding of software development life cycle (SDLC) and testing life cycle (STLC).
- Expertise in using test metrics to measure the effectiveness of testing.

## 4.2. Test Planning- Test Plan as per IEEE 829 STANDARD TEST PLAN TEMPLATE

### 1. Test Plan Identifier

- A unique name or code for the test plan so you can easily find and reference it.
- **Example:** *Test\_Plan\_Ecom\_App\_v1.0*

### 2. Introduction

- A short explanation about what is being tested and why.
- **Example:** *This test plan is for the online shopping app “ShopEasy.” The goal is to check if users can browse products, add to cart, and make payments smoothly.*

### 3. Test Items

- A list of what parts of the software will be tested.
- **Example:** *Product Search, Add to Cart, Checkout, Payment Integration.*

### 4. Features to be Tested

- The specific features or functions you will test.
- **Example:**
  - *Search by product name*
  - *Filter by price*
  - *Apply discount codes*

### 5. Features Not to be Tested

- Things you **won’t** test and why.
- **Example:** *We won’t test the “Wishlist” feature because it's still under development.*

### 6. Approach

- Your plan for how to test — which methods and levels of testing you’ll use.
- **Example:**
  - *Use **Black Box Testing** to check if the app works without looking at the code.*
  - *Perform **System Testing** to test the app as a whole.*

### 7. Item Pass/Fail Criteria

- What counts as a “pass” or “fail” for each test.
- **Example:** *A test case for the “Add to Cart” feature passes if the product count increases correctly after clicking the “Add to Cart” button.*

### 8. Suspension and Resumption Criteria

- When to **stop** testing and when to **start again**.
- **Example:** *Suspend testing if the app crashes repeatedly during payment. Resume testing once the development team fixes the issue.*

### 9. Test Deliverables

- The documents and reports you will create during testing.
- **Example:**
  - *Test Cases Document*

## *Bug Report*

- *Test Summary Report*

### **10. Testing Tasks**

- A list of tasks testers need to do.
- **Example:**
  - *Write test cases for login functionality*
  - *Run tests on mobile and web versions of the app*
  - *Report bugs in the defect tracking tool*

### **11. Environmental Needs**

- What tools, devices, and software are needed for testing.
- **Example:**
  - *Android and iOS smartphones for mobile testing*
  - *Test data for payment gateway testing*

### **12. Responsibilities**

- Who does what in the team.
- **Example:**
  - *Test Lead: Assign tasks and review bug reports*
  - *Tester: Execute test cases and log defects*

### **13. Staffing and Training Needs**

- What skills the team needs and any training they require.
- **Example:**
  - *Testers need knowledge of automation tools like Selenium.*
  - *Provide training on new bug tracking software.*

### **14. Schedule**

- A timeline showing when each testing task will happen.
- **Example:**
  - *Test case writing: March 10 - March 15*
  - *Test execution: March 16 - March 25*

### **15. Risks and Contingencies**

- Possible problems and backup plans.
- **Example:**

- *Risk: Internet issues during cloud testing.*
- *Contingency: Use offline simulators for mobile app testing*

## 16. Approvals

- The list of people who must approve the plan.
- **Example:**
  - *Project Manager*
  - *Test Lead*

IEEE 829 Template: [ieee-829.pdf](#)

## 4.3. Test Process Monitoring & Control

### 4.3.1. Test Monitoring through -Test Log (IEEE 829: TEST LOG TEMPLATE) and Defect Density

#### Test Log:

A test log is a simple record of all the tests done. It shows:

- Which test cases were run.
- Whether each test passed or failed.
- Any issues or bugs found.

This helps keep track of what has been tested and what still needs testing.

**1. Test Log Identifier :** Unique ID for the test log (e.g., TL\_GroceryApp\_001).

**2. Description :** Brief details about the test being performed, including the module or feature being tested.

**3. Date and Time :** The date and time when the test was executed.

**4. Tester's Name:** Name of the person conducting the test.

**5. Test Case ID :** The unique identifier for the test case being executed.

#### 6. Test Environment

- Device (e.g., Android/iOS)
- Operating System version
- Network (Wi-Fi/4G)

**7. Input Data :** Details of test inputs used (e.g., username, password, item selected).

**8. Expected Result :** The outcome that is expected based on requirements.

**9. Actual Result :** The actual outcome observed during testing.

**10. Status (Pass/Fail):** Indicates whether the test case passed or failed.

**11. Defect ID (if any):** If a bug is found, the defect ID from the defect tracking system is recorded.

**12. Remarks:** Additional comments, such as issues faced or improvements suggested.

Here are **3 simple test log examples** using the IEEE 829 format:

| Test Log ID | Date & Time          | Tester      | Test Case ID | Input Data                      | Expected Result          | Actual Result            | Status | Defect ID | Remarks                 |
|-------------|----------------------|-------------|--------------|---------------------------------|--------------------------|--------------------------|--------|-----------|-------------------------|
| TL_001      | 14-Mar-2025 10:00 AM | Priti Patil | TC_101       | User: test@mail.com, Pass: 1234 | Login successful         | Login failed             | Fail   | D_001     | Incorrect error message |
| TL_002      | 14-Mar-2025 10:15 AM | Priti Patil | TC_102       | User: user@mail.com, Pass: 0000 | "Invalid password" error | "Invalid password" error | Pass   | —         | —                       |
| TL_003      | 14-Mar-2025 10:30 AM | Priti Patil | TC_103       | Empty username and password     | "Fields required" error  | "Fields required" error  | Pass   | —         | —                       |

### Defect Density:

**Defect Density** is a software testing metric that measures the number of defects (bugs) found in a software module or component relative to its size (usually measured in lines of code or function points). It helps identify which parts of the software are more error

### Formula:

$$\text{Defect Density} = \frac{\text{Number of Defects Found}}{\text{Size of the Software Module (in KLOC or Function Points)}}$$

**KLOC:** Thousand Lines of Code

**Function Points:** A unit to measure software functionality

**Example:** Imagine you are testing two modules of an online grocery app:

- **Module A:** "User Registration" — has **5 defects** and **5000 lines of code** (5 KLOC)
- **Module B:** "Item Browsing" — has **8 defects** and **8000 lines of code** (8 KLOC)

**Defect Density for Module A:**

$$\text{Defect Density} = \frac{5}{5} = 1 \text{ defect per KLOC}$$

**Defect Density for Module B:**

$$\text{Defect Density} = \frac{8}{8} = 1 \text{ defect per KLOC}$$

#### 4.3.2. Reporting Test Status (IEEE 829: TEST SUMMARY REPORT TEMPLATE)

- 1. Test Summary Report Identifier:** A unique ID for the report (e.g., TSR\_GroceryApp\_001).
- 2. Test Summary:** A brief overview of the testing process, including what was tested, the modules covered, and the test objectives.
- 3. Variances:** Any differences between the planned testing and what was actually done. Example: "Login feature testing was delayed due to a server issue."

#### 4. Summary of Results

- **Total test cases executed:** 50
- **Passed:** 40
- **Failed:** 10
- **Blocked:** 2

This section highlights the overall test execution status.

- 5. Evaluation:** Assess the quality of the application based on test results:

"The application met 80% of the test objectives, but login and payment modules need further testing."



## 6. Summary of Activities A list of tasks performed:

- Test case design
- Test execution
- Defect reporting

7. **Approvals:** Sign-off section for stakeholders (Test Lead, Project Manager) to approve the test results.

## Example :

### 1. Test Summary Report Identifier: TSR\_GroceryApp\_001

### 2. Test Summary:

This Test Summary Report outlines the testing activities for the Online Grocery Ordering Mobile Application. The main functionalities tested include user registration, login, item browsing, price checks, and delivery time estimation.

### 3. Variances:

- **Planned:** Complete testing of all modules within 5 days.
- **Actual:** Testing delayed by 1 day due to server downtime during login module testing.

### 4. Summary of Results:

- **Total Test Cases Executed:** 50
- **Passed:** 40
- **Failed:** 8
- **Blocked:** 2

5. **Evaluation:** The application's core functionalities (registration, item browsing, and price checks) performed as expected. However, the **login** module encountered issues such as invalid error message handling. The **delivery time feature** showed inconsistent time estimates, requiring further testing.

### 6. Summary of Activities:

- **Test Planning:** Defined test scope, test objectives, and resources.
- **Test Case Design:** Created 50 test cases covering all key functionalities.
- **Test Execution:** Executed test cases and logged results.
- **Defect Reporting:** Reported 10 defects, tracked in the defect management tool.

### 7. Approvals:

- **Test Lead:** Mr Ram Patel (Signed)
- **Project Manager:** Mr. Praful More (Signed)

### 4.3.3 Test Control

**Test Control** is the process of continuously monitoring the test execution and making necessary adjustments to ensure that testing meets its objectives. It helps in managing risks, handling deviations from the test plan, and ensuring that the testing process runs smoothly.

Testing is a dynamic activity, and unexpected challenges like **delays, defects, or requirement changes** may arise. **Test Control** allows test managers to take corrective actions such as rescheduling tests, reallocating resources, or modifying test cases to align with the current situation.

#### Key Aspects of Test Control:

- **Tracking Progress :** Compare actual test execution against the test plan. Monitor test case execution (e.g., how many passed, failed, or blocked).
- **Defect Management :** Track defects and their resolution status. Prioritize critical defects for immediate fixes.
- **Re-planning & Adjustments:** If delays occur (e.g., server downtime), adjust the schedule. Modify test cases or scope if requirements change.
- **Risk Management:** Identify risks affecting testing (e.g., unstable test environment). Take actions to minimize risks (e.g., allocate additional resources).
- **Stakeholder Communication** Regular updates to Test Lead, Developers, and Project Managers. Daily or weekly reports on test progress and blockers.

#### Example

**Issue:** Login feature fails during testing due to a backend server issue.

- **Test Control Action:**
  - i. Report the defect to developers.
  - ii. Shift focus to other modules while the issue is fixed.
  - iii. Reschedule login tests once the server is stable.

#### 4.4. Test Scenario, Test Suite, Test Cases (both Positive & Negative Test Cases, as per IEEE 829: TEST CASE SPECIFICATION TEMPLATE)

##### Test Scenario

- A test scenario is a high-level idea of what needs to be tested.
- It is derived from requirements, use cases, or user stories.
- A test scenario helps testers think from an end-user perspective and ensure the system works as expected.

##### Why is a Test Scenario Important?

- Helps to cover all possible testing situations.
- Makes it easier to organize test cases.
- Saves time by reducing unnecessary test cases.
- Helps teams understand what needs to be tested in a structured way.

##### How to Write a Good Test Scenario?

- It should be clear and easy to understand.
- It should focus on the real-world use of the application.
- It should cover both positive and negative scenarios.
- It should be written in simple language so all team members can understand.

##### Example of a Test Scenario

example of a **login page** of a website. Some test scenarios could be:

1. Verify that the user can log in with **valid credentials**.
2. Check if an error message appears when **invalid credentials** are entered.
3. Ensure the user is **logged out successfully** when clicking the "Logout" button.
4. Verify the "Forgot Password" feature works correctly.
5. Ensure the system locks the account after multiple **wrong password attempts**

##### Test Suite

###### 1. What is a Test Suite?

A Test Suite is a collection of test cases that focus on testing a specific feature or module of an application. It helps organize test cases in a structured way. A test suite can contain both positive and negative test cases.

## Why is a Test Suite Important?

- Helps manage test cases efficiently.
- Ensures complete coverage of a feature or module.
- Makes it easier to track testing progress.
- Useful in automation testing for running multiple test cases together.

## How to Create a Good Test Suite?

- Group related test cases together.
- Cover both functional and non-functional aspects.
- Include positive and negative test cases.
- Organize in a way that is easy to track and execute.

Example of a **Login Module** of a website. A test suite for the **Login Feature** may include:

| Test Case ID | Test Case Description                                      |
|--------------|------------------------------------------------------------|
| TC001        | Verify login with valid username & password.               |
| TC002        | Verify error message for invalid credentials.              |
| TC003        | Check login with an empty username or password.            |
| TC004        | Verify the "Forgot Password" functionality.                |
| TC005        | Ensure account locks after multiple failed login attempts. |
| TC006        | Verify session expires after a period of inactivity.       |

## Test Case Template (IEEE 829 Format)

- **Test Case ID:** Unique identifier for the test case
- **Test Scenario:** General description of what is being tested
- **Preconditions:** Conditions that must be met before execution
- **Test Steps:** Step-by-step execution process
- **Expected Result:** The expected outcome after execution

- **Actual Result:** (To be filled during execution)
- **Status:** Pass/Fail (To be filled during execution)

1. **Positive Test Cases:** Positive test cases verify that the application works as expected under normal conditions by providing valid inputs To confirm that the system behaves correctly when given expected and valid inputs.
2. **Negative Test Cases:** Negative test cases check how the application behaves under invalid, incorrect, or unexpected conditions by providing invalid inputs. To ensure the system can handle errors, exceptions, and edge cases gracefully.

#### 4.5. Configuration Management- Configuration Management support for Testing

Configuration Management (CM) is a way to **keep track of changes** in a project. It helps manage updates to software, hardware, documents, and other important files so that everything stays **organized and correct**.

Think of it like **a library system**:

- Each book (software, document, or test case) has a specific version.
- If a book is updated (a new software version is released), the old version is stored properly so that people can refer back if needed.
- If someone wants to borrow a book (use a specific software version), they know exactly where to find it.
- If a mistake is found in a book (a bug in software), it is fixed and recorded properly so that everyone knows what was changed.

CM ensures that:

- Everyone works with the **correct version** of the software.
- Changes are **recorded** so nothing gets lost.
- Bugs and fixes are **tracked properly**.

#### Key Elements of Configuration Management in Testing

- **Version Control:**

Tracks different versions of test cases, test scripts, and test environments.  
Example: Using Git to manage test scripts.

➤ **Change Management:**

Manages changes in the software, ensuring they are tested properly before deployment.

Example: If a new feature is added, a tester gets notified and updates test cases accordingly.

➤ **Test Environment Management :**

Ensures that the right configurations (OS, database, browsers) are set up for testing.

Example: A tester needs to test a web app on Chrome, Firefox, and Edge. Configuration management ensures the correct browser versions are available

➤ **Test Case & Data Management**

Maintains different versions of test cases and test data for various test scenarios.

Example: A login test case for version 1.0 should not be mixed with version 2.0 test cases.

➤ **Defect Tracking & Reporting:**

Links test cases to defects, ensuring fixes are tested properly.

Example: If a bug is fixed, configuration management helps testers verify it in the right software version

### **Tools Used for Configuration Management**

- **Version Control Tools:** Git, SVN
- **Test Management Tools:** TestRail, QTest
- **Bug Tracking Tools:** JIRA, Bugzilla
- **Continuous Integration Tools:** Jenkins

## **4.6. Risk and Testing- Project Risk & Product Risk**

### **1. What is Risk in Testing?**

Risk in testing means the chance of something going wrong in a software project. It could be an issue that affects the project schedule, budget, or the quality of the final product.

There are **two main types of risks in testing**:

- 1. Project Risk** – Risks related to project management and execution.

**2. Product Risk** – Risks related to the quality and functionality of the software being developed.

### **Project Risk**

Project risk is any problem that **affects the project timeline, cost, or resources**. It is about the **things that can go wrong in managing the project**.

#### **Examples of Project Risks:**

- **Not enough time for testing** – The project has tight deadlines, and testing might not be completed.
- **Shortage of skilled testers** – If there are not enough experienced testers, the testing quality may suffer.
- **Changes in requirements** – If the client frequently changes what they want, it may delay testing.
- **Delays in development** – If developers take too long to complete the coding, testing gets less time.
- **Budget issues** – If there is not enough money, testing tools or resources might be limited.

#### **How to Handle Project Risks?**

- Plan extra time for testing.
- Use experienced testers.
- Communicate with developers to stay updated on changes.
- Use automation testing where possible to save time.

### **Product Risk**

Product risk is the chance of having defects or failures in the software. It is about the things that can go wrong in the software itself.

#### **Examples of Product Risks:**

- **Software crashes** – The system stops working suddenly.
- **Security vulnerabilities** – Hackers can steal user data.
- **Poor performance** – The application is too slow.
- **Incorrect calculations** – A banking app giving the wrong interest amount.
- **Compatibility issues** – A website that works on Chrome but not on Firefox.

#### **How to Handle Product Risks?**

- Identify high-risk areas and test them thoroughly.
- Perform **security testing** to check for vulnerabilities.
- Conduct **performance testing** to ensure the software runs smoothly.
- Do **cross-browser testing** to ensure compatibility.

### Difference between Product Risk and Project Risk

| Basis                    | Product Risk                                               | Project Risk                                             |
|--------------------------|------------------------------------------------------------|----------------------------------------------------------|
| <b>1. Meaning</b>        | Risk related to defects or quality of the software product | Risk related to planning and execution of the project    |
| <b>2. Focus</b>          | Focus on product features and functionality                | Focus on project process and management                  |
| <b>3. Impact</b>         | Affects product performance, security, and usability       | Affects time, cost, and project completion               |
| <b>4. Responsibility</b> | Handled by testers and developers                          | Handled by project managers and team leads               |
| <b>5. Occurrence</b>     | Occurs during development or testing phase                 | Occurs during planning and project execution             |
| <b>6. Examples</b>       | Bugs, system crash, wrong output, security flaws           | Delay in schedule, budget overrun, lack of resources     |
| <b>7. Detection</b>      | Identified during testing or after release                 | Identified during project tracking and monitoring        |
| <b>8. Solution</b>       | Fixed by debugging, testing, and improving code            | Managed by planning, scheduling, and resource allocation |

### 4.7. Incident/ Defect Management

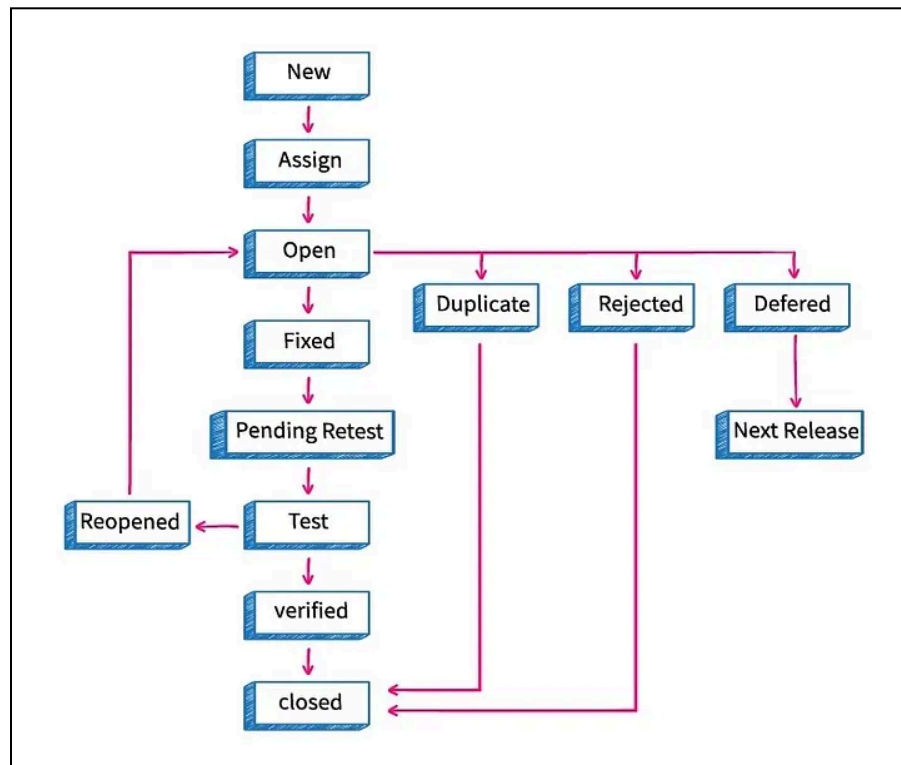
Incident and Defect Management is the process of identifying, tracking, and resolving defects (bugs) in a software system. The goal is to ensure that the software meets quality standards and functions correctly.

A **Defect** (or Bug) is a flaw in the software that causes incorrect results or unexpected behavior.

An **Incident** is any unexpected event that occurs during testing or in production, including defects, system crashes, or performance issues.



#### 4.7.1. Defect/ Bug Life Cycle



1. **New** – A defect is identified and recorded for the first time. It is logged into the defect tracking system with all details.No action is taken yet at this stage.
2. **Assign** – The defect is assigned to a developer or team member. Responsibility for fixing the issue is clearly defined. The developer reviews the defect details before starting work.
3. **Open** –The developer begins working on the defect.The issue is analyzed to find the root cause.Necessary changes are planned to fix it.
4. **Fixed** –The developer corrects the defect in the code.Required changes are implemented and updated. The defect is now ready for testing.
5. **Test** –The tester checks whether the defect is fixed properly. Different test cases are executed to validate the fix. The tester ensures no new issues are introduced.
6. **Verified** – The tester confirms that the defect is resolved successfully. The fix works as expected in all scenarios.The defect is ready to be closed.
7. **Closed** – The defect is completely resolved and finalized. No further testing or changes are required. It is marked as completed in the system.
8. **Reopened** –If the defect still exists, it is opened again. The previous fix did not work correctly. It is sent back to the developer for correction.

9. **Duplicate** –The defect is already reported earlier. It matches an existing defect in the system. So, it is marked as duplicate to avoid repetition.
10. **Rejected** – The defect is not valid or cannot be reproduced. It may be due to misunderstanding or incorrect data. So, it is not considered for fixing.
11. **Deferred** –The defect is valid but not fixed immediately. It is postponed due to low priority or time constraints. It will be handled in future if needed.
12. **Next Release** – The defect is planned to be fixed in the next version.It is not critical for the current release. It will be resolved in upcoming updates.

#### **4.7.2. Defect/ Incident Report (IEEE 829: TEST INCIDENT REPORT TEMPLATE )**

##### **Contents of a Test Incident Report:**

1. **Incident Report ID:** A unique identifier for the defect.
2. **Summary:** A short description of the defect.
3. **Test Environment:** Details of the system where the defect occurred (OS, browser, version, etc.).
4. **Steps to Reproduce:** The exact steps to follow to see the defect again.
5. **Expected Result:** What should have happened if there was no defect.
6. **Actual Result:** What actually happened due to the defect.
7. **Severity:** How serious the defect is (Critical, High, Medium, Low).
8. **Priority:** How urgently the defect needs to be fixed (High, Medium, Low).
9. **Attachments:** Screenshots, logs, or videos to help understand the defect.
10. **Status:** Current state of the defect (New, Assigned, Fixed, Closed, etc.).

## Defect/ Incident Report for Grocery Ordering Mobile Application

**Incident Report ID:**DEF-001

**Summary:** "Add to Cart" button is not functioning on the product details page.

### Test Environment:

- **Device:** Samsung Galaxy S21
- **OS:** Android 12
- **App Version:** 2.1.5
- **Network:** Wi-Fi & Mobile Data

### Steps to Reproduce:

1. Open the Grocery Ordering mobile application
2. Search for any product (e.g., "Milk 1L").
3. Tap on the product to open the details page.
4. Click on the "Add to Cart" button.

**Expected Result:** The product should be added to the shopping cart, and a success message should appear.

**Actual Result:** Nothing happens when the "Add to Cart" button is clicked. The product is not added to the cart.

**Severity:** High (Users cannot place orders properly)

**Priority:** High (Needs to be fixed immediately)

### Attachments:

Screenshot of the product page with the non-functional button.

Screen recording showing the issue.

**Status:** New